

2.c.Unrolled List

–Delete Element

–Space Complexity Analysis and Algorithm

Program:

```
void deleteElement(int val) {
    if (head == NULL) {
        cout << "The list is empty. Nothing to delete.\n";
        return;
    }

    struct Node* temp = head;
    struct Node* prev = NULL;

    while (temp != NULL) {
        for (int i = 0; i < temp->numElements; i++) {
            if (temp->elements[i] == val) {

                // Element found: Shift remaining elements left
                for (int j = i; j < temp->numElements - 1; j++) {
                    temp->elements[j] = temp->elements[j + 1];
                }
                temp->numElements--;

                cout << "Element " << val << " deleted
successfully.\n";

                // Check if the node is now completely empty
                if (temp->numElements == 0) {
                    if (prev == NULL) {
                        head = temp->next;
                        free(temp);
                    } else {
                        prev->next = temp->next;
                        free(temp);
                    }
                    cout << "An empty node was removed from the
list.\n";
                }
            }
        }
    }
}
```

```
        return;
    }
}
prev = temp;
temp = temp->next;
}

cout << "Element " << val << " not found in the list.\n";
}
....
    case 2:
        cout << "Enter integer value to delete: ";
        cin >> value;
        deleteElement(value);
        break;
```

Space Complexity Analysis

1. Input Space Analysis

The Input Space is the memory required to store the data that the algorithm operates on.

We can look at this from two perspectives:

- *the specific parameter passed to the function,*
- *and the entire data structure it acts upon.*

A. Parametric Input Space

This is the memory required to, store the explicit arguments passed directly into the function call.

Code: void deleteElement(int val)

Analysis: The function takes exactly one parameter: a single integer val. This integer requires a fixed, constant amount of memory (typically 4 bytes), regardless of how large the linked list is.

Complexity: $O(1)$ (Constant Space)

B. Full Data Holistic Input Space

This is the memory required to store the entire existing unrolled linked list that the function is searching through and modifying.

Analysis: *The data structure consists of N nodes. Each struct Node contains:*

- *An array of fixed capacity B ($MAX_ELEMENTS = 4$)*
- *An integer ($numElements$)*
- *A pointer ($*next$)*

Because each node holds a constant maximum of B elements (where $B = 4$), the memory size of a single node is fixed.

To store n total elements, the structure requires $N \approx \left\lceil \frac{n}{B} \right\rceil$ nodes.

The size of one node is constant ($O(1)$). Because there are N nodes in the list, the total memory consumed by the entire data structure scales linearly with the number of elements n .

Complexity: $O(N)$ or $O(n)$ (Linear Space)

2. Auxiliary Space Complexity: $O(1)$

Auxiliary Space measures the extra, temporary memory that the algorithm uses while executing, exclusively excluding the input data itself. This determines how much additional RAM that our function demands to do its job.

*Let's break down every temporary variable created inside the **deleteElement** block:*

- **Pointers:** The function initializes two local pointer variables to traverse the list:
 - `struct Node * temp = head;`
 - `struct Node * prev = NULL;`
- **Loop Counters:** The function uses standard integer variables for iterating:
 - `int i` for the search loop.
 - `int j` for the shifting loop.

Key Observations:

1. **No Dynamic Allocation:** The function never calls `malloc`, `new`, or creates any new nodes. It only modifies or frees existing memory (`free(temp)`).
2. **No Recursion:** The function uses iterative `while` and `for` loops. There are no recursive function calls, meaning the Call Stack does not grow. It stays at a fixed depth of 1 frame.
3. **Constant Memory:** The pointers (`temp, prev`) and integers (`i, j`) take a fixed amount of bytes.

Result: Because the extra memory required by the function is fixed and completely independent of the input size `n` or `N`, the Auxiliary Space Complexity is strictly $O(1)$ (Constant Space).

For Academic Answer = Auxiliary Space = $O(1)$.

"As we often cannot control the size of the input data, but we can control how much extra memory of our algorithm creates to process it."

Total Space Complexity or Total memory footprint:

***= Input Space (Parametric Input Space + Full Data Holistic Input Space)
+ Auxiliary Space***

= $(O(1) + O(N)) + O(1)$

= $O(N)$.

or,

$(O(1) + O(n)) + O(1)$

= $O(n)$.

Extra:

When, dynamically memory allocation to cause $O(N)$ auxiliary space, the amount of memory requested must scale up as the size of the list(N) scales up.

1. Looping N times to allocate N nodes.

```
// If a function did this, it would take  $O(N)$  auxiliary space
for (int i = 0; i < N; i++) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    // ... link the node ...
}
```

1. The Loop Execution

The for loop is governed by the condition $i < N$, meaning it will execute exactly N times.

2. The Dynamic Memory Allocation

*Inside the loop, the **malloc** function is called during every single iteration.*

***malloc** physically claims a brand – new block of memory from the computer's heap.*

3. Total Memory Consumed

*If **sizeof(struct Node)** takes a fixed constant amount of memory (for example, 24 bytes), the total extra memory generated by this loop is:*

$$\text{Total Memory} = N \times 24 \text{ bytes.}$$

*In Big – O notation, we drop the exact hardware constants (like 24 bytes) because we only care about how the memory requirements grow as N gets larger. Because the memory requested scales linearly with the variable N , the complexity is **$O(N)$** .*

2. Allocating a massive block of memory in a single line:

```
struct Node* array_of_nodes = (struct Node*)malloc(N * sizeof(struct Node));
```

*Even though malloc is only called one time here,
the total size of the memory requested is multiplied by N.
If N gets larger, the memory block gets larger.*

$$\text{Space Complexity} = O(N)$$

Note: We came through this scenario earlier in

skip list:

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level + 1));
```

if we have wrote:

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * N);
```

*here ,space complexity would have been : **$O(N)$***

But as it is: $(level + 1) = O(level)$ and level is capped with a hardcoded constant:

```
#define MAX_LEVEL 3
```

As it require `level + 1` , we can say space complexity = $O(level)$, but :

As, level = MAX_LEVEL = 3 is a constant ,

The function allocates dynamic memory on the heap.

Because $level \leq 3$, the forward array will allocate a maximum of 4 pointers.

Allocating a fixed maximum amount of memory, is $O(1)$ space.

Algorithm: UnrolledDELElem(val)

1. [CHECK IF LIST IS EMPTY]

- **IF $START = NULL$, then:**
 - **Write: "The list is empty. Nothing to delete."**
 - **RETURN**
- **[End of If structure.]**

2. [INITIALIZE TRAVERSAL POINTERS]

- **SET $TEMPPTR := START$**
- **SET $PREVPTR := NULL$**

3. [OUTER LOOP: NODE TRAVERSAL]

- **REPEAT WHILE $TEMPPTR \neq NULL$:**

4. [INNER LOOP: ELEMENT SEARCH]

- **SET $i := 0$**
- **REPEAT WHILE $i < numElements[TEMPPTR]$:**

5. [CHECK FOR MATCH AND DELETE]

- **IF $elements[TEMPPTR][i] = val$, Then:**
 - **Write: "Element ", val , " deleted successfully."**

(Shift remaining elements left)

- **SET $j := i$**
- **REPEAT WHILE $j < numElements[TEMPPTR] - 1$:**
 - **SET $elements[TEMPPTR][j] := elements[TEMPPTR][j + 1]$**
 - **SET $j := j + 1$**

[End of Shifting while Loop]

(Decrement element count)

▪ *SET numElements[TEMPPTR] := numElements[TEMPPTR] - 1*

(Check if the node is now completely empty)

▪ *IF numElements[TEMPPTR] = 0, Then:*

▪ *IF PREVPTR = NULL, Then:*

▪ *SET START := LINK[TEMPPTR]*

▪ *ELSE:*

▪ *SET LINK[PREVPTR] := LINK[TEMPPTR]*

[End of inner If structure.]

▪ *FREE(TEMPPTR) (Return node space to memory)*

▪ *Write: "An empty node was removed from the list."*

▪ *[End of empty node check If structure.]*

▪ *RETURN (Exit the algorithm immediately)*

▪ *[End of Match If structure.]*

▪ *SET i := i + 1*

○ *[End of Inner Search Loop]*

(Move to the next node if not found in the current node)

○ *SET PREVPTR := TEMPPTR*

○ *SET TEMPPTR := LINK[TEMPPTR]*

• *[End of Outer Node Traversal Loop]*

6. [ELEMENT NOT FOUND]

• *Write: "Element ", val, " not found in the list."*

7. END OF ALGORITHM
